| Mark | |
|---|---|

| | |
|---|---|
| **Team name** | *A10* |
| **Homework number** | *HOMEWORK 04* |
| **Due date** | 16/10/2026 |

| **Contribution** | NO | Partial | Full |
|---|---|---|---|
| Carbone Alessandro | | | *X* |
| Casareale Alberto | | | *X* |
| Colombi Federico | | | *X* |
| Lorusso Luca | | | *X* |
| Martorano Gabriele | | | *X* |
| Notes: Everyone contributed to understanding the logic and implementing the code | | | |

| **Project name** | 4A - Send data to PC | | |
|---|---|---|---|
| Not done | Partially done (major problems) | Partially done (minor problems) | Completed |
| | | | *X* |

**Explanation:**

Part 1: Hardware Configuration and Routing

To achieve the goal of sending information from the microcontroller to the PC every second using the USART interface and DMA, we configured the peripherals via STM32CubeMX. We enabled USART2 in asynchronous mode for communication over the Virtual COM port, utilizing the default assigned pins. To allow background transfers without blocking the CPU, we configured and activated the DMA for UART data transmission (Tx), which was assigned to the DMA1_Stream6 channel. We also configured Timer 1 (TIM1) to be used as a time base. To ensure proper execution, we enabled the necessary flags within the NVIC settings: we checked the update interrupt for Timer 1 (TIM1 Update interrupt) and the global interrupt for USART2, in addition to the DMA interrupt (DMA1_Stream6_IRQn) in order to trigger the automatic end-of-transmission routines.

Part 2: Code Generation and Verification

After generating the project code, we verified the timer settings in the main.c file. For Timer 1, which we use to keep track of time, we set the prescaler (PSC) to 84-1 and the counter period (ARR) to 1000-1. This configuration, assuming an 84 MHz system clock, sets the base frequency of Timer 1 to 1 kHz, thus generating a "tick" (interrupt) exactly every millisecond. To handle data transmission, we initialized the following global variables: a character buffer msg of size 64, an integer length for the string size, a count counter for milliseconds, and a boolean flag is_transmitting to prevent overlapping or multiple sends during the DMA transfer.

Part 3: Functional Logic Implementation

Finally, we implemented the system's functional logic based on callback functions. Time counting is triggered at startup by calling the HAL_TIM_Base_Start_IT(&htim1) function before the infinite loop. The passage of time is managed by the HAL_TIM_PeriodElapsedCallback routine: every time Timer 1 generates an interrupt (every millisecond) and there are no transmissions already in progress (is_transmitting == false), the count counter is incremented. When count reaches 1000 (i.e., exactly 1 second), the program uses the snprintf function to insert the name and year ("Group A10 %d\r\n", 2026) into the buffer and save its length. Immediately, it sends the message to the PC by calling the HAL_UART_Transmit_DMA function, sets the is_transmitting = true flag, and resets the counter to 0. Once the DMA has physically finished transferring the string byte by byte, the system automatically triggers the end-of-transmission interrupt handled by the HAL_UART_TxCpltCallback function, inside which we restore is_transmitting = false to unlock the system and put it in standby for the next second.

**Professor comments:**

| **Project name** | 4B - Receive commands from PC | | |
|---|---|---|---|
| Not done | Partially done (major problems) | Partially done (minor problems) | Completed |
| | | | *X* |

**Explanation:**

Part 1: Hardware Configuration and Routing

To achieve the goal of receiving variable-length text commands from the PC via UART to control the turning on and off of the green LED, we configured the peripherals in STM32CubeMX. We configured USART2 in asynchronous mode (TX/RX) to enable bidirectional communication on the Virtual COM port. To manage the communication efficiently without burdening the processor, we configured the DMA for both UART data reception (Rx), associated with the DMA1_Stream5 channel, and UART data transmission (Tx), associated with the DMA1_Stream6 channel. To control the LED, we left the associated pin (LD2) configured in output mode (GPIO_OUTPUT). To ensure the system handles interrupts correctly, we enabled the USART2 global interrupt, the EXTI line interrupt (EXTI15_10_IRQn for the button), and the global interrupts related to the DMA (DMA1_Stream5_IRQn and DMA1_Stream6_IRQn) within the NVIC.

Part 2: Code Generation and Verification

After generating the project code, we set up the necessary global variables in the main.c file. We defined a constant RX_SIZE equal to 50 to allocate a reception buffer rx (which will hold the strings sent from the PC), a transmission buffer tx of size 100 for the responses to be sent back to the PC, and an integer variable length for formatting. Inside the main(), right before entering the infinite while(1) loop, we "armed" the system by putting it in listening mode: to do this, we used the suggested function HAL_UARTEx_ReceiveToIdle_DMA(&huart2, (uint8_t *)rx, sizeof(rx)). This function is crucial because it

allows receiving a variable number of characters, stopping the acquisition as soon as the data line enters an idle state.

Part 3: Functional Logic Implementation

Finally, we implemented the functional logic for command handling within the HAL_UARTEx_RxEventCallback interrupt routine. This callback is triggered automatically as soon as the transmission from the PC stops. Inside it, to prevent any potential buffer overflow, the code first checks if the received data size equals or exceeds the buffer capacity; if so, it safely truncates the size. Then, it manually appends a null terminator '\0' at the correct index to ensure the buffer becomes a valid string. Immediately after, to improve reliability, we inserted a for loop that automatically converts any lowercase letters to uppercase, making the command case-insensitive. Using the strcmp function, we then check the string's content: if the buffer contains exactly "LED ON", the microcontroller forces the LD2 pin high (turning on the LED) and loads the confirmation "The LED is now ON\r\n" into the tx buffer. If the string matches "LED OFF", the pin is pulled low (turning off the LED) and the respective turn-off message is prepared. If a different command is sent, the code generates an "Unknown command" warning message. In conclusion, the response message is sent to the PC using HAL_UART_Transmit_DMA, thus offloading the transmission in the background to the DMA controller, and immediately after, the HAL_UARTEx_ReceiveToIdle_DMA function is called again to virtually clear the buffer and put the UART back to listening for the next command.

**Professor comments:**